Pakka Note: Yani Call Spectator

Review of the current repo and build specification for Google AI Studio

Prepared 20 Oct 2026. Repo reviewed: amisha-0709/pkka_note_GAS (main). Challenge deadline: 10 Nov 2026, 11:59 PM IST.

1. How to use this document

This document gives the building agent the full picture: the end vision, what the repo does today, where it differs from the vision, and exactly what to build. Section 15 is a master prompt that can be pasted into Google AI Studio. Sections 2 to 14 are the reference it points to, so paste or attach the whole document.

Review basis. I read the README, package.json, server.ts, the repo file listing and your Pakka Note explainer. GitHub blocked automated access to the src/ folder and individual source files, so statements about code inside src/ come from the README and explainer and are marked *verify in code*. Nothing here claims that tests, builds or live Gnani calls were run.

2. End vision

A support agent is on a live call with a customer. The customer shares unique details: name, phone number, email ID, device identifier (IMEI or serial number), order or ticket numbers, and similar. **Yani is a silent spectator.** It listens to the call, spots those details, captures them accurately, and builds call notes in real time. The agent uses the notes and captured details to fill forms, tickets and CRM records.

One-line product statement: Yani listens so the agent can focus on the customer. It never speaks on the call, it proposes, and the agent decides.

Non-negotiable rules for Yani

1. **Never speaks into the call.** No audio output is ever mixed into or played on the call channel. Yani has no voice during a call.
2. **Never talks to the customer.** All output goes to the agent's screen only.
3. **Never silently finalizes.** Every captured value is a candidate until the agent confirms or edits it. This keeps the original Pakka Note principle: speech recognition proposes, a human confirms.

4. **Always traceable.** Every captured value links to the transcript moment where it was heard, with timestamp and speaker.
5. **Honest about confidence.** Format validity is not proof of correctness or ownership. The UI must say so plainly.
6. **Privacy first.** Raw audio is not stored by default. Sensitive numbers (card, Aadhaar, PAN, banking PIN, OTP) are detected, masked and never stored.

3. What the repo is today

Area	Current state
Concept	Voice-to-structured-data verification: capture one spoken identifier, transcribe, validate, **read it back with TTS**, confirm or correct, audit.
Stack	React 19, TypeScript, Vite, Tailwind 4, Motion, Lucide on the front end. Express 4, multer, tsx and esbuild on the back end. @google/genai is installed but, per the README, not used for any feature.
Speech	Gnani Prisma STT (api.vachana.ai/stt/v3, modes transcribe and verbatim, en-IN and hi-IN) and Gnani Timbre TTS (timbre-v2.5, voice Nalini).
Routes	/api/health, capture, readback, confirmation, correction, records, benchmark, testConnection.
Fields	Mobile (10 digits, starts 6 to 9), postal PIN, service tag (7 chars, NATO phonetics), spelled name (experimental).
UI tabs	Capture, History, Benchmark, Settings. Capture shows a 4-way diagnostic: what user said, what Prisma heard, what FieldEngine parsed, validation.
Persistence	Container-local JSON at data/pakka_note_audit.json with atomic writes and rollback on flush failure.
Tests	node:test suites for FieldEngine, state machine, mocked-provider route tests.

Area	Current state
Claims	README says Prisma/Timbre error contracts and confirmation routes are route-tested with mocked providers, and a tone smoke test hit the live provider. Live speech accuracy is pending.

What is good and should be kept: the candidate-not-truth philosophy, the FieldEngine parsing ideas (spoken digits, double and triple, Hindi numerals, NATO phonetics), the Prisma and Timbre adapters with timeouts and error classes, the safe error handler in server.ts (it redacts long tokens and never leaks stacks), server-side secrets, the audit trail with atomic writes, and the honest evidence matrix.

4. Gap analysis: current repo versus the vision

The current app is a **single-person, single-utterance, voice-in/voice-out confirmation tool**. The vision is a **passive, continuous, two-party call listener that outputs notes**. Several core assumptions have to change.

#	Current behaviour	Vision requires	Severity
1	Yani speaks: Timbre reads the value back to the user.	Yani never speaks on a call. Read-back becomes a **text prompt on the agent's screen** that the agent can say to the customer.	Critical, contradicts the vision
2	Input is one uploaded or recorded clip per capture (10 MB limit).	Continuous live listening for the whole call, with partial results appearing within seconds.	Critical
3	User picks one field type and one language before capturing.	Yani auto-detects every entity type as it is spoken, with no pre-selection.	Critical
4	One speaker. No concept of agent versus customer.	Two speakers. Customer details matter most. Agent repeats and read-backs are confirmation evidence, not new data.	Critical

#	Current behaviour	Vision requires	Severity
5	Output is one validated candidate.	Output is **call notes**: captured details, issue summary, actions, follow-ups, open items.	Critical
6	Confirmation is a spoken yes or no classified by the state machine.	The **agent** confirms with one click. Customer confirmation heard on the call is a supporting signal only.	High
7	Fields: mobile, PIN, service tag, name.	Add **email** and **device ID (IMEI, serial)**; add order and ticket IDs as configurable patterns; promote name from experimental.	High
8	No handling of mid-call corrections like 'no, 4 not 5'.	Detect and apply spoken corrections, keep full value history.	High
9	No sensitive-data guard.	Detect card, Aadhaar, PAN, OTP, banking PIN. Mask them and never store.	High
10	Gemini SDK installed but unused.	Use Gemini for entity spotting in context, correction detection, summary and notes, always followed by deterministic validation.	High
11	UI is a four-tab diagnostic tool.	A call workspace: live transcript, live captured-details panel, live notes, post-call review.	High
12	README says 'Zero customer PII retained', yet the audit log stores validated candidates such as phone numbers.	Make the claim true or reword it. Define retention and add a purge control.	Medium

#	Current behaviour	Vision requires	Severity
13	package.json name is react-example, a bun.lock sits beside npm scripts, and .aistudio/artifacts is committed.	Rename the package, keep one package manager, remove the AI Studio artifacts folder.	Low
14	README mentions pre-packaged audio fixtures. *Verify in code* that they exist.	Provide labelled synthetic two-speaker call scripts and clips for demo and evaluation.	Medium
15	Benchmark measures single-utterance accuracy.	Evaluate extraction precision and recall on scripted two-speaker calls, clean and noisy.	Medium
16	JSON file store is ephemeral.	Put storage behind an interface so it can move to Firestore later. Fine for the demo.	Low

5. Target architecture

```
Agent browser
  Mic (agent) ──────────┐
  Customer audio ───────┤  two separate channels, 16 kHz mono, chunked 3 to 6
s
  (tab/system audio     |
   or file/simulation)  ▼
                POST /api/calls/:id/chunk  (channel = agent | customer)
                       |
Express server         ▼
  1. Prisma STT per chunk (mode transcribe, plus verbatim for numeric-heavy
chunks)
  2. Transcript store: timestamped segments, speaker from channel
  3. Entity pipeline:
       a. FieldEngine deterministic parsers (digits, email, IMEI, names, IDs)
       b. Gemini JSON extraction with a strict schema over a rolling window
       c. Merge, validate, de-duplicate, apply corrections, mask sensitive
data
  4. Notes engine (Gemini): rolling summary, then final notes at call end
  5. SSE stream  GET /api/calls/:id/events  ──► agent UI updates live
                       |
  Store (interface): calls, segments, candidates, notes, audit events
```

Speaker attribution. The most reliable approach is channel separation: agent microphone is one stream, customer audio is another. Browsers cannot read the phone line directly, so for the demo support three input modes: (a) agent mic plus shared tab or system audio carrying the customer, (b) upload of a two-channel or two-file recording, (c) a built-in **Simulated call** that plays a scripted customer and agent conversation. Mode (c) matters most for the challenge demo because it is repeatable and uses only synthetic data. If only a mixed mono source exists, run Gemini-based speaker labelling as a fallback and mark the labels as inferred.

Streaming caveat. The repo uses Prisma v3 as a request-response API. Check Gnani's documentation for a streaming or WebSocket option and prefer it if it exists. If not, use short overlapping chunks (3 to 6 s with about 1 s overlap, cut at silences where possible) and de-duplicate overlapping words. Target: speech to on-screen text in under 4 seconds. Record chunk latency in the audit log.

Role of Timbre. Not used on the call. Keep the adapter. Optional and off by default: a private 'whisper to agent' read-back played only to the agent's own headset output, never on the call channel, behind a clear toggle. If kept, label it clearly in the UI. Decide before

the demo whether showing Timbre is worth the risk of appearing to contradict the spectator rule.

6. Feature specification

P0: required for the submission demo

1. **Call session lifecycle.** Start call (consent banner shown), live status, end call, review. States: idle, listening, paused, ended, reviewed.
2. **Live two-speaker transcript.** Agent and customer turns, timestamps, partial text, entity spans highlighted inline.
3. **Auto entity detection** for: person name, mobile number, email, device ID (IMEI and serial), order or ticket ID, postal PIN. No field selection by the user.
4. **Captured Details panel.** One card per entity with: value, type, status, confidence, evidence quote, timestamp, speaker, and actions Confirm, Edit, Dismiss, Copy.
5. **Candidate lifecycle:** detected, needs_review, confirmed, edited, dismissed, superseded. Never auto-confirm.
6. **Spoken-correction handling.** 'No, it is 4 not 5', 'sorry, last digit is 7', 'let me repeat' create a new candidate that supersedes the old one. History stays visible.
7. **Agent read-back helper (text only).** Each card shows a pre-formatted read-back line such as '98765 43210' with a Copy button, so the agent can say it to the customer. When the customer says yes on the call, show a soft 'customer verbally confirmed' tag, which still requires the agent's click to finalize.
8. **Live call notes.** A notes panel updated every 20 to 30 seconds: summary so far, issue, details captured, actions promised, follow-ups, open items.
9. **Post-call notes.** Final structured notes, editable, with Copy, Download .md and Download .json.
10. **Sensitive-data guard.** Detect card numbers (Luhn), Aadhaar-like 12 digits, PAN pattern, OTP and PIN phrases. Show a masked chip such as 'Card number detected, masked, not stored'. Never place them in notes.
11. **Simulated call mode** with at least three scripted calls: clean, noisy or telephony-like, and one with a correction.
12. **Audit trail.** Events: call started, chunk processed, candidate created, confirmed, edited, dismissed, notes generated, call ended. Values in audit follow the retention setting.

13. **Honest status bar.** Shows Prisma and Gemini connectivity, current latency, and a mode badge: Live, Simulated, Offline.

P1: strengthens the entry

1. Language handling for English, Hindi and Hinglish code-mixed speech, with per-chunk language detection or a language toggle.
2. Confidence scoring from: validation passed, value repeated consistently, customer confirmed on call, STT verbatim and transcribe agreement.
3. Verbatim versus transcribe cross-check for numeric chunks, flagging disagreement as 'needs review'.
4. Evaluation tab running scripted calls and reporting precision, recall and exact-match per entity type, clean versus noisy, with sample counts.
5. Customizable field templates: the agent chooses which details to require, and the UI shows a checklist of captured versus missing.
6. Export to a CRM-style JSON payload and a stub 'Send to CRM' that clearly says it is a demo.
7. Keyboard shortcuts: C confirm, E edit, D dismiss.

P2: only if time remains

Open-items reminders, sentiment signal, call-quality hint when audio is poor, multi-call history search, Firestore storage.

7. Entity types and rules

Every entity has a deterministic parser and validator. Gemini may *suggest* an entity, but a value is only shown as valid if the deterministic validator accepts it.

Entity	Spoken forms to handle	Validation	Notes
Mobile	'nine eight seven...', 'double nine', 'triple five', Hindi digits (ek, do, teen, char, paanch, chhe, saat, aath, nau, shunya, double), '+91' or leading 0	10 digits, starts 6 to 9 after stripping prefix	Reuse existing FieldEngine logic. Group as 5-5 in read-back.

Entity	Spoken forms to handle	Validation	Notes
Email	'at', 'at the rate', 'dot', 'underscore', 'dash' or 'hyphen', 'gmail dot com', spelled letters, NATO words	RFC-lite regex, plausible TLD, no spaces	Low STT reliability, so always needs_review. Support 'spelled as' sequences.
Device ID, IMEI	15 digits, spoken in groups	15 digits plus **Luhn check**	Luhn gives strong real validation.
Device ID, serial	Alphanumeric, NATO phonetics, 'zero' versus 'O'	8 to 20 alphanumeric, configurable	Flag ambiguous O/0, I/1, S/5, B/8.
Name	Spoken name plus optional spelling ('R as in Romeo A H U L')	2 to 40 letters	Prefer the spelled version when present.
Order or ticket ID	Configurable regex per template, such as ORD followed by 8 digits	Pattern match	Simple settings screen.
Postal PIN	Six digits	First digit 1 to 9	Demo data only.

Collision rule: a 10-digit number could be a phone or part of something else. Use context words ('my number', 'IMEI', 'order') from the surrounding transcript window to choose the type, and show the second-best type as an alternative on the card.

8. Data model (TypeScript)

```
type Speaker = 'agent' | 'customer' | 'unknown';
type EntityType = 'name' | 'mobile' | 'email' | 'imei' | 'serial' |
'order_id' | 'postal_pin' | 'custom';
type CandidateStatus = 'detected' | 'needs_review' | 'confirmed' | 'edited' |
'dismissed' | 'superseded';

interface Call {
  id: string; startedAt: string; endedAt?: string;
  mode: 'live' | 'simulated' | 'upload';
  state: 'idle' | 'listening' | 'paused' | 'ended' | 'reviewed';
```

```
  language: 'en-IN' | 'hi-IN' | 'mixed';
  consentShown: boolean; templateId?: string;
}

interface Segment {
  id: string; callId: string; speaker: Speaker; speakerInferred: boolean;
  startMs: number; endMs: number; text: string; verbatimText?: string;
  final: boolean; sttLatencyMs: number;
}

interface Candidate {
  id: string; callId: string; type: EntityType;
  value: string; displayValue: string;        // displayValue may be masked
  status: CandidateStatus; confidence: number; // 0..1
  validation: { passed: boolean; checks: string[]; warnings: string[] };
  evidence: { segmentId: string; quote: string; speaker: Speaker; atMs:
number };
  supersedes?: string; customerVerbalConfirm?: boolean;
  alternativeType?: EntityType; createdAt: string; resolvedAt?: string;
resolvedBy?: 'agent';
}

interface CallNotes {
  callId: string; version: number; final: boolean;
  summary: string; issue: string;
  details: { candidateId: string; label: string; value: string }[];
  actions: string[]; followUps: string[]; openItems: string[];
  generatedAt: string; model: string;
}

interface AuditEvent { id: string; callId: string; type: string; at: string;
meta: Record<string, unknown>; }
```

9. API specification

Method and path	Purpose
POST /api/calls	Create a call. Body: mode, language, templateId. Returns call.

Method and path	Purpose
POST /api/calls/:id/chunk	multipart: audio, channel (agent or customer), seq, startMs. Runs STT and extraction. Returns 202 quickly.
GET /api/calls/:id/events	**Server-Sent Events** stream: segment, candidate, notes, status events.
POST /api/calls/:id/pause and /resume and /end	Lifecycle. End triggers final notes.
PATCH /api/calls/:id/candidates/:cid	Body action: confirm, edit (with value), dismiss. Re-validates on edit.
GET /api/calls/:id	Full call with segments, candidates, notes.
POST /api/calls/:id/notes/regenerate	Regenerate notes from current state.
GET /api/calls/:id/export?format=md or json	Download notes.
DELETE /api/calls/:id	Purge all data for the call.
POST /api/simulate/:scriptId	Replays a scripted call into the pipeline with realistic timing.
GET /api/health	Keep, extend with Gemini status and last latency.

Keep the existing JSON error shape (success, errorCode, message) and the global safe error handler. Retire or repurpose /api/readback, /api/confirmation, /api/correction as described in section 10.

10. What to change in the existing code

Keep and extend: fieldEngine.ts (add email, IMEI with Luhn, serial, name-with-spelling, order ID), gnaniPrisma.ts (add chunked use and overlap handling), environment.ts and health route, persistence layer (wrap in a storage interface), server.ts error handling, and the test style.

Rework: confirmationStateMachine.ts becomes a candidate lifecycle machine driven by agent actions. The spoken yes/no classifier stays only to set the soft 'customer verbally confirmed' tag. The correction helper (replace last N characters, replace at index) can power the agent's Edit action and the spoken-correction handler.

Demote: gnaniTimbre.ts and /api/readback. Keep the adapter, but the main flow never calls it. Optional private whisper only.

Remove or replace: CapturePage.tsx (becomes LiveCallPage), the field and language pickers, the single-clip upload flow, the 4-way diagnostic as the main view (keep a compact developer version under Settings).

Add: call and segment stores, extraction pipeline, Gemini client (server-side only, GEMINI_API_KEY from AI Studio secrets), notes engine, SSE hub, sensitive-data guard, simulator, scripted call fixtures.

Housekeeping: rename package, pick npm or bun, delete .aistudio/artifacts, add a GitHub Actions workflow running npm test, npm run lint and npm run build, update README claims to match reality.

11. UI specification

Layout (desktop first, responsive down to tablet): three columns during a call.

1. **Left: Live Transcript.** Speaker-labelled bubbles, newest at the bottom, auto-scroll with a pause-on-hover. Entities underlined by type colour. Click a highlight to jump to its card.
2. **Centre: Captured Details.** Cards stacked newest first, grouped by status. Each card shows type icon, value in large monospace, confidence bar, status chip, evidence quote with timestamp and speaker, validation checks as small ticks, the copy-ready read-back line, and buttons Confirm, Edit, Dismiss. Superseded values collapse under the active one.
3. **Right: Live Notes.** Sections for Summary, Issue, Actions, Follow-ups, Open items, plus a checklist of required details (captured or missing) from the template.

Top bar: call timer, state, mode badge (Live, Simulated, Offline), Prisma and Gemini health dots, latency, Start or Pause or End, and a persistent 'Yani is listening. It does not speak on this call.' indicator.

Screens: Home (start live call, start simulated call, past calls), Live Call, Post-Call Review (editable final notes, all captured details, transcript, export), Evaluation, Settings (templates, retention, sensitive-data rules, developer diagnostics).

Visual tone: calm, high-contrast, professional support-desk feel. Large legible monospace for values so agents can read them at a glance. Never use red for a normal state. Use a distinct amber for needs_review. Support dark and light. Accessible: keyboard operable,

ARIA live region for new captured details, no colour-only meaning.

Empty, loading and error states are required: no mic permission, no customer audio shared, STT failure for a chunk (show a gap marker in the transcript, never fabricate text), Gemini unavailable (fall back to deterministic extraction only and say so), offline.

12. Call notes generation

The notes engine runs on a rolling window and again at call end, using Gemini with a strict JSON schema. A current Gemini Flash model is a good default, configurable through a GEMINI_MODEL environment variable.

Rules for the notes prompt:

1. Use only what is in the transcript. Do not invent facts, names, numbers or commitments.
2. Attribute commitments to the right speaker ('Agent will send replacement by Friday').
3. Captured details in notes come from the Candidate store and show status. Unconfirmed values are labelled 'unconfirmed'.
4. Never include masked sensitive data.
5. Keep the summary to 3 to 5 sentences, actions and follow-ups as short bullets.
6. If something is unclear, put it under Open items instead of guessing.

The notes are editable by the agent. Regeneration never overwrites agent edits without confirmation.

13. Privacy, security and honesty

1. Show a consent banner before listening starts. Require the agent to acknowledge that the customer has been informed per their organization's policy.
2. Do not store raw audio. Process chunks in memory and discard.
3. Retention setting for transcripts and values: session only, 24 hours, or until deleted. Default to session only for the demo. Provide DELETE per call and a purge-all button.
4. API keys (GNANI_API_KEY, GEMINI_API_KEY) stay server-side in AI Studio Secrets, never in client code, logs, screenshots or the repo. Keep masking in health output.
5. Sensitive-data guard as in section 6, item 10.
6. Demo only with synthetic or explicitly consented voices and fake identifiers.

7. Wording rules: 'valid format' not 'verified', 'captured' not 'confirmed' until the agent confirms, and never claim a number is active or owned by the caller.
8. Never present simulated results as live Gnani accuracy or noise robustness. Label every metric with mode, sample count and conditions.

14. Test, evaluation and acceptance

Unit tests: every parser with spoken-form variants, IMEI Luhn pass and fail, email assembly, correction supersession, sensitive-data masking, candidate lifecycle transitions.

Route tests with mocked providers: chunk to candidate flow, SSE events delivered in order, edit re-validates, dismiss and purge, Gemini failure fallback, STT failure creates a gap marker and no fake text.

Scripted evaluation set: at least 10 synthetic two-speaker calls with known ground truth for names, phones, emails, IMEIs and corrections. Run each clean and with telephony-like degradation (8 kHz band-limit, noise). Report precision, recall and exact match per entity type, plus end-to-end latency. Report failures as failures.

Acceptance checks for the demo:

1. A simulated call shows live transcript, at least four captured details and live notes without any user action beyond Start.
2. A spoken correction supersedes the earlier value and keeps history.
3. An invalid value (wrong-length phone, IMEI failing Luhn) appears as needs_review with a clear reason.
4. A card number spoken in the call is masked and absent from notes and storage.
5. No audio is ever played by Yani during a call.
6. Final notes can be edited and exported as .md and .json.
7. A Gnani or Gemini outage degrades gracefully, with the mode badge showing it.
8. npm test, npm run lint and npm run build pass, and results are recorded.

15. Master prompt to paste into Google AI Studio

> You are upgrading an existing app called Pakka Note (repo amisha-0709/pkka_note_GAS: React 19 + TypeScript + Vite + Tailwind front end, Express back end with Gnani Prisma STT and Timbre TTS adapters, FieldEngine parsers, JSON audit store). Read the attached specification document in full and follow it.
>
> **Goal:** turn the app from a single-utterance voice read-back tool into **Yani, a silent call spectator**. A support agent is on a live call with a customer. Yani listens to both sides, detects unique details the customer shares (name, mobile, email, device ID such as IMEI or serial, order or ticket ID), captures them as candidates with evidence, and builds live call notes. Yani never speaks on the call and never talks to the customer. The agent confirms, edits or dismisses every captured value. The agent can read values back to the customer using a copy-ready text line on screen.
>
> **Build in this order, finishing and testing each phase before the next:**
>
> 1. **Foundation.** Rename the package, remove .aistudio/artifacts and the duplicate lockfile, add a storage interface and the Call, Segment, Candidate, CallNotes and AuditEvent types from section 8. Keep tests green.
> 2. **Entity engine.** Extend FieldEngine with email, IMEI (Luhn), serial, name with spelling, order ID and the sensitive-data guard. Add a candidate lifecycle machine with spoken-correction supersession. Full unit tests.
> 3. **Pipeline.** Implement POST /api/calls, /chunk, SSE events, pause, resume and end. Use Prisma per chunk with overlap de-duplication. Add Gemini JSON extraction (server-side, strict schema) merged with deterministic validation. Gemini suggestions that fail validation must never appear as valid.
> 4. **Notes engine.** Rolling and final notes per section 12 with Gemini, editable, exportable.
> 5. **UI.** Build the three-column Live Call screen, Post-Call Review, Home, Settings, and the Evaluation tab per section 11. Include all empty and error states.
> 6. **Simulator.** Add three scripted two-speaker synthetic calls (clean, noisy, with a correction) that replay with realistic timing through the same pipeline and are labelled Simulated everywhere.
> 7. **Hardening.** Privacy controls, retention, purge, consent banner, health status, route tests with mocked providers, GitHub Actions workflow, README rewritten to match reality.

> **Hard rules:** never play audio during a call; never auto-confirm; never store raw audio; never expose API keys to the client; never fabricate transcript text when STT fails; never describe format validity as verification; label every metric as simulated or live. API keys are GNANI_API_KEY and GEMINI_API_KEY from server-side Secrets, with an optional GEMINI_MODEL variable.
>
> After each phase, run npm test, npm run lint and npm run build, and report real outputs. Ask me before making any decision that is listed as open in section 16.

16. Open questions and assumptions

1. **Customer audio source.** For the demo, is tab or system audio sharing acceptable, or should the challenge demo rely on Simulated and uploaded two-channel recordings only? Assumption: Simulated plus tab audio.
2. **Prisma streaming.** Does Gnani offer a streaming or WebSocket STT endpoint for v3? Assumption: no, so chunked requests are used. Needs a check against Gnani's documentation.
3. **Prisma on Hinglish and noisy telephony audio.** Unproven. Measure before claiming anything. This ties to the noise robustness angle in your explainer.
4. **Timbre.** Keep as an optional private whisper to the agent, or drop it from the demo? Assumption: optional and off by default.
5. **Gemini role.** Assumed acceptable alongside Gnani, since the build is in AI Studio. Confirm the challenge terms allow it and that Gnani remains the primary speech layer.
6. **Storage.** JSON store is fine for the demo. Move to Firestore only if you need history to survive restarts.
7. **Timeline.** About 21 days to the deadline. Suggested split: phases 1 to 3 in the first week, phases 4 to 5 in the second, phases 6 to 7 plus demo video and submission in the third.